01 / 처음 보는 분을 위한 3분 설명

코드를 잘 찾아주면, AI의 답도 더 정확해질까?

symbol_index와 계층 요약 방식의 차이를 작은 코드 실험으로 확인했습니다.



코드·함수

작업을 수행하는 조리법

예: 가격을 계산하는 규칙

symbol_index

조리법의 이름과 위치를 적은 주소록

필요한 원문으로 빠르게 이동



계층 요약

조리법 → 묶음 → 전체의 요약 노트

여러 파일의 관계를 함께 탐색

같은 질문을 했을 때의 정답률

현재 주소록 방식

16.7%

A · 원문을 찾아 답변

계층 요약 방식

18.8%

RH · 원문 + 요약에서 답변

차이

+2.1%p

퍼센트포인트 차이

서로 다른 48문제 중 현재 방식은 8개, 계층 요약 방식은 9개를 맞혔습니다.

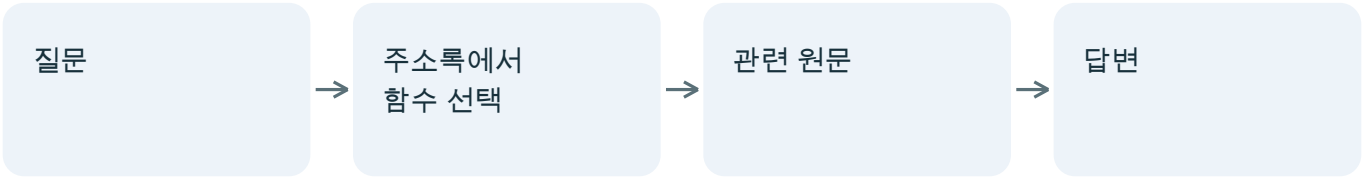
큰 숫자 옆에 꼭 필요한 조건

작은 합성 Python 프로젝트 6개, 서로 다른 문제 48개, 문제당 3회 반복의 결과입니다. 탐색적 95% 구간은 -8.3~+12.5%p입니다. 통계적으로 우위가 확정된 결과는 아닙니다.

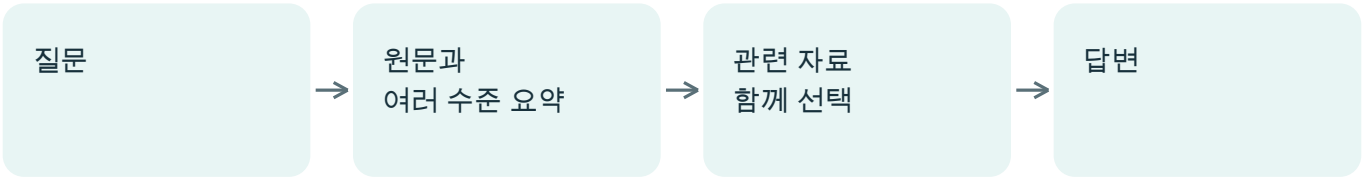
코드의 동작과 요청한 답변 순서를 모두 맞혀야 정답입니다. 실제 제품의 버그를 고치거나 개발 시간을 줄이는지까지 확인한 결과는 아닙니다.

02 / 왜 달라지는가

주소록만 바꾼 것이 아니어서, 요약의 효과를 따로 확인했습니다



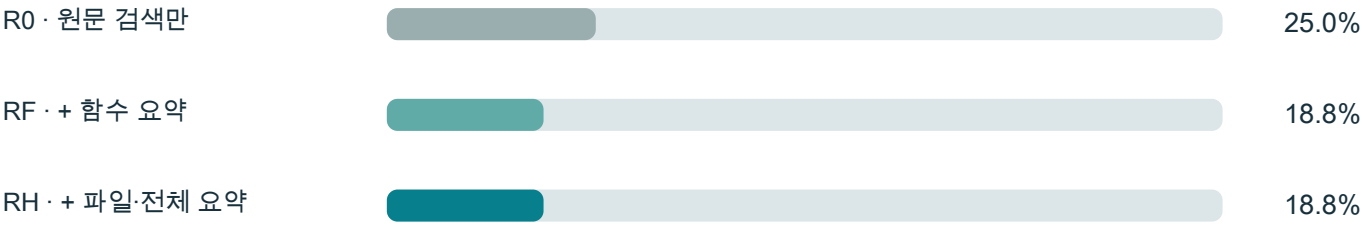
현재 방식 A



계층 요약 RH · 요약은 원문으로 돌아갈 수 있는 출처와 연결

요약만 추가하면?

같은 원문 검색에 계층 요약을 더하자 주 정답률이 6.2%p 낮아졌습니다. 출력 순서 오류만 제외한 사후 보조 점수는 두 방식 모두 25.0%입니다.



위 세 방식은 같은 검색 방법과 최대 자료 길이를 사용합니다. 차이는 요약의 추가 여부입니다.

차이가 가장 컸던 질문

'호출 구조'에서 RH - A는 -33.3%p였습니다. 유형별 6~12문제의 탐색적 결과입니다.

RAPTOR의 계층 요약 아이디어를 코드에 맞게 적용했습니다. 원논문의 모든 기술을 재현한 것은 아닙니다.

03 / 그래서 무엇을 선택할까

정답률뿐 아니라 준비 시간과 적용 범위를 함께 봅니다

질문 하나의 평균 시간 · A

2.35초

검색 + 모델의 선택 + 답변

질문 하나의 평균 시간 · RH

4.22초

검색 + 답변

요약 노트를 만드는 준비 시간

평가 코드 6개 프로젝트의 최초 요약 준비에 경과시간 합계 9.5분이 들었습니다. 코드가 바뀌면 요약을 고치는 비용도 생길 수 있지만, 이번에는 측정하지 않았습니다.

이번 결과가 제안하는 다음 행동

이번 조건에서는 RH의 정답률이 높고 A가 빨랐습니다. 최고 정답률도 25.0%인 파일럿입니다. 요약 없이 파일 간 원문을 잇는 B도 25.0%였고, 3.17초로 같은 점수인 R0의 4.26초보다 빨랐습니다. B를 실제 저장소에서 먼저 검증합니다.

확실히 기억할 세 가지

01 숫자는 이 시험의 성적표

48개 작은 코드 문제에서 얻은 결과이며 모든 프로젝트의 성능은 아닙니다.

02 요약은 틀리거나 빠뜨릴 수 있음

출처가 연결되어 있어도, 실제 원문 근거가 충분히 들어왔는지는 별도로 확인해야 합니다.

03 실제 도입은 다음 검증 후

실제 저장소·코드 수정·변경 후 요약 갱신까지 시험해야 합니다.

자세한 표·실패 사례·실험 설정은 함께 제공한 상세보고서에 있습니다. 모든 결과의 원본은 같은 폴더의 evidence에 저장했습니다.